Übungsblatt 1 – Stellenwertsysteme

Grundlagen Informatik | DHBW

Aufgabe 1 – Grundbegriffe

(a) Erläutern Sie den Unterschied zwischen *analogen* und *digitalen* Signalen. Was verarbeitet ein Digital-Rechner grundsätzlich?

(b) Definieren Sie den Begriff *Stellenwertsystem* und erklären Sie, welche Bedeutung die Basis B hat. Welche Ziffern sind in einem System mit Basis B erlaubt?

(c) Erklären Sie die Begriffe *Bit*, *Byte* und *MSB* (Most Significant Bit) im Kontext des Dualsystems.

Aufgabe 2 – Binär $\leftrightarrow$ Dezimal *(Berechnung)*

(a) Wandeln Sie die folgenden Dualzahlen in das Dezimalsystem um, indem Sie die Stellenwerte explizit aufsummieren:

(i) 10110110_2
(ii) 11001010_2
(iii) 01111111_2

(b) Wandeln Sie die folgenden Dezimalzahlen in das Dualsystem um. Verwenden Sie die *fortgesetzte Division durch 2* und notieren Sie alle Zwischenschritte:

(i) 182_{10}
(ii) 75_{10}
(iii) 255_{10}

(c) Was ist der größtmögliche Dezimalwert, der mit 8 Bit (1 Byte) dargestellt werden kann? Begründen Sie Ihre Antwort.

Aufgabe 3 – Hexadezimalsystem *(Berechnung)*

(a) Das Hexadezimalsystem verwendet 16 Ziffern. Welche Symbole werden für die Werte 10 bis 15 verwendet und warum?

(b) Wandeln Sie $\texttt{BAD}_{16}$ in das Dezimalsystem um. Zeigen Sie den Rechenweg mit Stellenwerten.

(c) Wenden Sie den *Taschenrechner-Trick* (fortgesetzte Division mit Dezimalrest-Methode) an, um 2989_{10} in das Hexadezimalsystem umzurechnen.

(d) Wandeln Sie $\texttt{BAD}_{16}$ in das Dualsystem um, indem Sie jede Hex-Ziffer einzeln in 4-Bit-Gruppen zerlegen.

(e) Nennen Sie fünf verschiedene Schreibweisen für die Hexadezimalzahl BAD. Geben Sie jeweils an, in welchem Kontext (Programmiersprache, Assembler etc.) die Notation üblich ist.

Aufgabe 4 – Gemischte Basiskonversion *(Szenario)*

Ein Datenbankadministrator findet in einem Speicherauszug (Memory-Dump) folgende Werte:

Wert (gegeben)	Gesucht
$\mathtt{FF}_{16}$	Dezimal und Binär
$1010\,0001_2$	Dezimal und Hexadezimal
163_{10}	Binär und Hexadezimal
$\mathtt{2A}_{16}$	Dezimal und Binär

(a) Führen Sie alle verlangten Konversionen durch und zeigen Sie den Rechenweg.

(b) Beschreiben Sie, welcher Konversionspfad (Dez→Bin, Bin→Hex usw.) bei welcher Aufgabe am effizientesten ist und warum.

Aufgabe 5 – Vorsilben & Speichergrößen

(a) Füllen Sie folgende Tabelle aus. Geben Sie jeweils den genauen Dezimalwert sowie die zugehörige Zweierpotenz an:

Vorsilbe	Dezimal (SI)	Binär (IEC)	IEC-Name
Kilo	$10^?$	$2^?$	?
Mega	$10^?$	$2^?$	?
Giga	$10^?$	$2^?$	?
Tera	$10^?$	$2^?$	?
Peta	$10^?$	$2^?$	?
Exa	$10^?$	$2^?$	?

(b) Ein Hersteller bewirbt eine Festplatte mit *„1 TB“*. Festplattenhersteller verwenden dabei das **SI-Kilo** (10^3). Ein Betriebssystem zeigt die Kapazität jedoch in **binären** Einheiten an. Wie viel GiB zeigt das Betriebssystem tatsächlich an? Geben Sie den Rechenweg an.

(c) Ein RAM-Riegel hat 4 GiB Kapazität. Wie viele Byte sind das exakt? Wie viele Bit?

Aufgabe 6 – Wahr oder Falsch

Beurteilen Sie die folgenden Aussagen als *wahr* oder *falsch* und begründen Sie jeweils kurz:

(a) Im Dualsystem gibt es nur die Ziffern 0 und 1, weil Computer mit zwei verschiedenen Spannungspegeln (An/Aus) arbeiten.

(b) Die Hexadezimaldarstellung $\mathtt{FF}_{16}$ entspricht dem Dezimalwert 255.

(c) Eine Binärzahl mit 8 Bit kann genau 8 verschiedene Werte darstellen.

(d) Beim Umrechnen von Binär nach Hexadezimal werden immer *3* aufeinanderfolgende Bits zu einer Hex-Ziffer zusammengefasst.

(e) 1 KiB (Kibibyte) ist exakt dasselbe wie 1 KB (Kilobyte).

(f) Das *Most Significant Bit* (MSB) ist das Bit mit dem größten Stellenwert und steht ganz links in der Binärdarstellung.

Lösungen – Übungsblatt 1

Aufgabe 1 – Grundbegriffe

(a) **Analoge Signale** sind kontinuierlich und können jeden beliebigen Wert innerhalb eines Wertebereichs annehmen – etwa die physikalische Spannung in einem Kabel. **Digitale Signale** hingegen sind *diskret*: Sie nehmen nur endlich viele, klar unterscheidbare Werte an (z. B. genau zwei Spannungspegel für 0 und 1). Ein Digital-Rechner verarbeitet intern ausschließlich *Zahlen* (kodierte Bitmuster) zusammen mit *Operatoren* (arithmetischen und logischen Befehlen). Die gesamte Informationsverarbeitung – Text, Bild, Ton – wird auf diese numerische Darstellung reduziert.

(b) Ein **Stellenwertsystem** zur Basis B stellt Zahlen als gewichtete Summe dar, wobei die n-te Stelle (von rechts, beginnend mit $n = 0$) das Gewicht B^n trägt. Die Basis B gibt dabei an, wie viele verschiedene Ziffern das System kennt: erlaubt sind genau die ganzzahligen Werte $\{0, 1, \ldots, B-1\}$. Für $B = 10$ (Dezimalsystem) sind das die vertrauten Ziffern 0–9; für $B = 2$ (Dualsystem) sind es nur 0 und 1. Ein Beispiel im Dezimalsystem: $2078_{10} = 2 \cdot 10^3 + 0 \cdot 10^2 + 7 \cdot 10^1 + 8 \cdot 10^0$.

(c) Ein **Bit** (Binary Digit) ist die kleinste Informationseinheit und kann genau die Werte 0 oder 1 annehmen. Acht Bit bilden gemeinsam **1 Byte**, das die gängigste Basiseinheit der Datenmenge ist. Das **MSB** (Most Significant Bit, höchstwertiges Bit) ist dasjenige Bit, das den größten Stellenwert trägt – bei einer 8-Bit-Zahl entspricht das dem Stellenwert $2^7 = 128$ und steht ganz links in der Binärdarstellung. Entsprechend ist das **LSB** (Least Significant Bit) ganz rechts mit Stellenwert $2^0 = 1$.

Aufgabe 2 – Binär ↔ Dezimal

(a) **Binär → Dezimal** (Stellenwerte aufsummieren):

(i) 10110110_2:

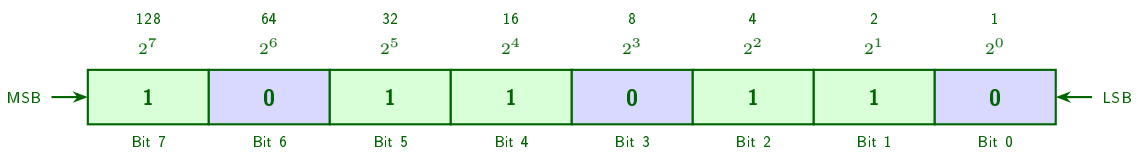
$$1{\cdot}128 + 0{\cdot}64 + 1{\cdot}32 + 1{\cdot}16 + 0{\cdot}8 + 1{\cdot}4 + 1{\cdot}2 + 0{\cdot}1 = 128 + 32 + 16 + 4 + 2 = \mathbf{182_{10}}$$

(ii) 11001010_2:

$$1{\cdot}128 + 1{\cdot}64 + 0{\cdot}32 + 0{\cdot}16 + 1{\cdot}8 + 0{\cdot}4 + 1{\cdot}2 + 0{\cdot}1 = 128 + 64 + 8 + 2 = \mathbf{202_{10}}$$

(iii) 01111111_2:

$$0{\cdot}128 + 1{\cdot}64 + 1{\cdot}32 + 1{\cdot}16 + 1{\cdot}8 + 1{\cdot}4 + 1{\cdot}2 + 1{\cdot}1 = 64 + 32 + 16 + 8 + 4 + 2 + 1 = \mathbf{127_{10}}$$



Beispiel: $10110110_2 = 182_{10}$ (grüne Felder $= 1$, blaue $= 0$)

(b) **Dezimal** → **Binär** (fortgesetzte Division durch 2, Reste *von unten lesen*):

(i) 182_{10}:

$$182 \div 2 = 91 \text{ R } \mathbf{0}, \quad 91 \div 2 = 45 \text{ R } \mathbf{1}, \quad 45 \div 2 = 22 \text{ R } \mathbf{1}, \quad 22 \div 2 = 11 \text{ R } \mathbf{0},$$

$$11 \div 2 = 5 \text{ R } \mathbf{1}, \quad 5 \div 2 = 2 \text{ R } \mathbf{1}, \quad 2 \div 2 = 1 \text{ R } \mathbf{0}, \quad 1 \div 2 = 0 \text{ R } \mathbf{1} \Rightarrow \mathbf{10110110_2}$$

(ii) 75_{10}:

$$75 \div 2 = 37 \text{ R } \mathbf{1}, \quad 37 \div 2 = 18 \text{ R } \mathbf{1}, \quad 18 \div 2 = 9 \text{ R } \mathbf{0}, \quad 9 \div 2 = 4 \text{ R } \mathbf{1},$$

$$4 \div 2 = 2 \text{ R } \mathbf{0}, \quad 2 \div 2 = 1 \text{ R } \mathbf{0}, \quad 1 \div 2 = 0 \text{ R } \mathbf{1} \Rightarrow \mathbf{1001011_2}$$

(iii) 255_{10}:

$$255 \div 2 = 127 \text{ R } 1, \; 127 \div 2 = 63 \text{ R } 1, \; 63 \div 2 = 31 \text{ R } 1, \; 31 \div 2 = 15 \text{ R } 1,$$

$$15 \div 2 = 7 \text{ R } 1, \; 7 \div 2 = 3 \text{ R } 1, \; 3 \div 2 = 1 \text{ R } 1, \; 1 \div 2 = 0 \text{ R } 1 \Rightarrow \mathbf{11111111_2}$$

(c) Mit 8 Bit lassen sich $2^8 = 256$ verschiedene Bitmuster kodieren. Der kleinste Wert ist $00000000_2 = 0$, der größte ist $11111111_2 = 255 = 2^8 - 1$. Damit beträgt der größtmögliche Dezimalwert **255**. Dies folgt direkt aus der Summe aller Stellenwerte: $128+64+32+16+8+4+2+1 = 255$.

Aufgabe 3 – Hexadezimalsystem

(a) Da das Hexadezimalsystem die Basis $B = 16$ hat, werden 16 Ziffernsymbole benötigt. Die Ziffern 0–9 entsprechen den gewohnten Dezimalwerten. Für die Werte 10–15 werden die Großbuchstaben **A=10, B=11, C=12, D=13, E=14, F=15** verwendet. Das ist nötig, weil ein einzelnes Zeichen einen eindeutigen Stellenwert repräsentieren muss – würde man z. B. „10“ schreiben, wäre unklar, ob zwei Ziffern (1 und 0) oder ein Wert (zehn) gemeint ist.

(b) $\texttt{BAD}_{16}$ mit Stellenwerten ($16^2 = 256$, $16^1 = 16$, $16^0 = 1$):

$$\texttt{B} \cdot 256 + \texttt{A} \cdot 16 + \texttt{D} \cdot 1 = 11 \cdot 256 + 10 \cdot 16 + 13 \cdot 1 = 2816 + 160 + 13 = \mathbf{2989_{10}}$$

(c) **Taschenrechner-Trick** (2989_{10} → Hexadezimal): Zunächst klassische Division durch 16 für die erste Stelle:

$$2989 \div 16 = 186{,}8125 \quad \Rightarrow \quad \text{Ganzzahlquotient: } 186, \quad \text{Rest: } 0{,}8125 \times 16 = \mathbf{13} = \texttt{D}$$

$$186 \div 16 = 11{,}625 \quad \Rightarrow \quad \text{Ganzzahlquotient: } 11, \quad \text{Rest: } 0{,}625 \times 16 = \mathbf{10} = \texttt{A}$$

$$11 \div 16 = 0{,}6875 \quad \Rightarrow \quad \text{Ganzzahlquotient: } 0, \quad \text{Rest: } 0{,}6875 \times 16 = \mathbf{11} = \texttt{B}$$

Reste von unten lesen: $\texttt{BAD}_{\mathbf{16}}$ ✓

(d) **Hex** → **Binär** (jede Hex-Ziffer = 4 Bit):

$$\texttt{B} = 11_{10} = 1011_2, \quad \texttt{A} = 10_{10} = 1010_2, \quad \texttt{D} = 13_{10} = 1101_2$$

$$\texttt{BAD}_{16} = \underbrace{1011}_{B}\ \underbrace{1010}_{A}\ \underbrace{1101}_{D}{}_{\,2} = \mathbf{101110101101_2}$$

Diese 1:4-Entsprechung macht Hex zu einer kompakten Kurzschreibweise für lange Bitmuster.

(e) **Fünf Schreibweisen für die Hexadezimalzahl BAD:**

(i) `$BAD` – Der Dollar-Prefix ist die klassische Konvention in Assemblersprachen wie 6502-ASM und Motorola 68k.

(ii) `0xBAD` – Das Prefix `0x` ist der Standard in C, C++, Java, Python und vielen weiteren Hochsprachen.

(iii) `BAD_h` bzw. `BADh` – Das Suffix `h` wird in Intel-x86-Assembler (NASM, MASM) verwendet; bei führendem Buchstaben wird eine `0` vorangestellt (`0BADh`).

(iv) `BAD_Hex` bzw. $\texttt{BAD}_{\texttt{Hex}}$ – Diese ausgeschriebene Tiefstellung findet sich in Lehrbüchern und mathematischer Notation, um die Basis unmissverständlich klarzumachen.

(v) $\texttt{BAD}_{16}$ bzw. $\texttt{BAD}_{16}$ – Die rein numerische Tiefstellung mit der Basiszahl 16 ist die in Mathematik und Informatik-Vorlesungen üblichste Schreibweise.

Die Vielfalt der Notationen entsteht, weil unterschiedliche Werkzeuge (Compiler, Assembler, Taschenrechner) jeweils eigene Konventionen entwickelt haben. Entscheidend ist stets, dass der Leser die Basis eindeutig erkennen kann.

Aufgabe 4 – Gemischte Basiskonversion

(a) Alle Konversionen im Überblick:

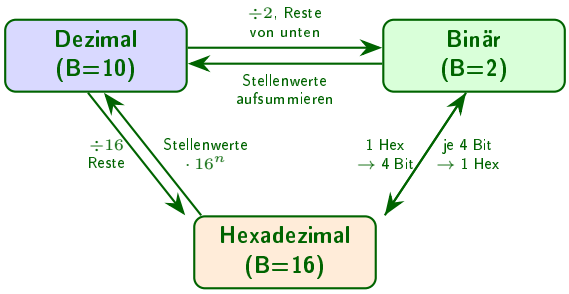
$\texttt{FF}_{16}$: $\texttt{F} \cdot 16 + \texttt{F} \cdot 1 = 15 \cdot 16 + 15 = 240 + 15 = \mathbf{255_{10}}$; $\texttt{F} = 1111_2 \Rightarrow \mathbf{11111111_2}$.

10100001_2: $128 + 32 + 1 = \mathbf{161_{10}}$; Gruppen: $\underbrace{1010}_{A}\underbrace{0001}_{1} \Rightarrow \texttt{A1}_{\mathbf{16}}$.

163_{10}: $163 = 128 + 32 + 2 + 1 = \mathbf{10100011_2}$; Gruppen: $\underbrace{1010}_{A}\underbrace{0011}_{3} \Rightarrow \texttt{A3}_{\mathbf{16}}$.

$\texttt{2A}_{16}$: $2 \cdot 16 + 10 = 32 + 10 = \mathbf{42_{10}}$; $2 = 0010_2$, $\texttt{A} = 1010_2 \Rightarrow \mathbf{00101010_2}$.

(b) Der effizienteste Pfad hängt vom Ausgangssystem ab: **Binär** → **Hex** geht am schnellsten durch das Zusammenfassen von je 4 Bits zu einer Hex-Ziffer – kein Rechenaufwand, nur Ablesen. **Hex** → **Binär** funktioniert analog durch Auffalten jeder Hex-Ziffer. Für **Dezimal** → **Binär** ist die fortgesetzte Division durch 2 systematisch; **Dezimal** → **Hex** nutzt Division durch 16. Den Umweg Dezimal → Binär → Hex vermeidet man, wenn eine direkte Hex-Division möglich ist.



Aufgabe 5 – Vorsilben & Speichergrößen

(a) Ausgefüllte Tabelle der Vorsilben:

Vorsilbe	Dezimal (SI)	Dezimalwert	Binär (IEC)	IEC-Name
Kilo	10^3	1 000	$2^{10} = 1\,024$	Kibi (KiB)
Mega	10^6	1 000 000	$2^{20} = 1\,048\,576$	Mebi (MiB)
Giga	10^9	1 000 000 000	$2^{30} = 1\,073\,741\,824$	Gibi (GiB)
Tera	10^{12}	$1{,}0 \times 10^{12}$	$2^{40} = 1\,099\,511\,627\,776$	Tebi (TiB)
Peta	10^{15}	$1{,}0 \times 10^{15}$	$2^{50} = 1\,125\,899\,906\,842\,624$	Pebi (PiB)
Exa	10^{18}	$1{,}0 \times 10^{18}$	$2^{60} = 1\,152\,921\,504\,606\,846\,976$	Exbi (EiB)

(b) **Festplatten-Kapazität in GiB:**
Der Hersteller gibt 1 TB im SI-Sinne an: $1\,\text{TB} = 10^{12}$ Byte. Das Betriebssystem rechnet in GiB, wobei $1\,\text{GiB} = 2^{30}$ Byte $= 1\,073\,741\,824$ Byte. Daher:

$$\frac{10^{12}\ \text{Byte}}{2^{30}\ \text{Byte/GiB}} = \frac{1\,000\,000\,000\,000}{1\,073\,741\,824} \approx \mathbf{931{,}3}\ \text{GiB}$$

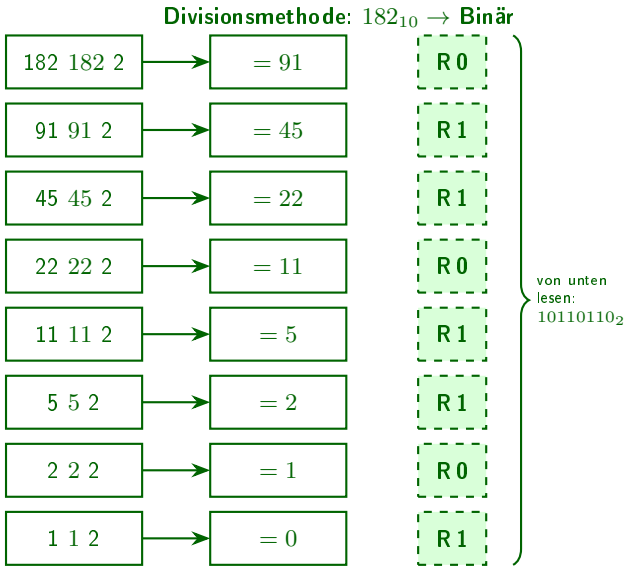
Das Betriebssystem zeigt also etwa **931 GiB** an – der scheinbare „Verlust" von rund 70 GB entsteht allein durch die unterschiedliche Definition der Präfix-Einheiten.

(c) $4\,\text{GiB}$ in Byte und Bit:

$$4\,\text{GiB} = 4 \times 2^{30}\ \text{Byte} = 2^2 \times 2^{30}\ \text{Byte} = 2^{32}\ \text{Byte} = \mathbf{4\,294\,967\,296}\ \text{Byte}$$

$$\times\ 8\,\text{Bit/Byte} = 2^{32} \times 2^3 = 2^{35}\ \text{Bit} = \mathbf{34\,359\,738\,368}\ \text{Bit}$$

Aufgabe 6 – Wahr oder Falsch



(a) **Wahr.** Computer-Hardware ist auf zwei stabile Zustände ausgelegt (Transistor gesperrt/leitend, entspricht 0/1). Das Dualsystem ist daher das natürliche Zahlensystem der Digitalelektronik – jeder andere Zustand wäre fehleranfällig und schwieriger herzustellen.

(b) **Wahr.** $\text{FF}_{16} = 15 \cdot 16 + 15 \cdot 1 = 240 + 15 = 255_{10}$. Gleichzeitig gilt $11111111_2 = 2^8 - 1 = 255$, was zeigt, dass FF_{16} den maximal darstellbaren Wert eines Bytes entspricht.

(c) **Falsch.** Mit n Bit lassen sich 2^n verschiedene Werte darstellen, nicht n. 8 Bit erlauben also $2^8 = 256$ Werte (0 bis 255). Die Aussage verwechselt die Anzahl der Bits mit der Anzahl der darstellbaren Zustände – ein typischer Denkfehler.

(d) **Falsch.** Es werden immer **4** Bits (nicht 3) zu einer Hex-Ziffer zusammengefasst. Das liegt daran, dass man mit 4 Bits genau $2^4 = 16$ verschiedene Werte kodieren kann – exakt so viele, wie das Hexadezimalsystem Ziffern hat (0–F).

(e) **Falsch.** $1\,\text{KiB}$ (Kibibyte) ist exakt $2^{10} = 1\,024$ Byte, während $1\,\text{KB}$ (Kilobyte) im SI-System genau $10^3 = 1\,000$ Byte bedeutet. Der Unterschied beträgt 24 Byte pro Einheit und summiert sich bei großen Datenmengen erheblich auf. Die IEC-Präfixe (Kibi, Mebi, Gibi…) wurden eingeführt, um diese Mehrdeutigkeit zu beseitigen.

(f) **Wahr.** Das MSB (Most Significant Bit) trägt den höchsten Stellenwert (2^{n-1} bei einer n-Bit-Zahl) und steht in der konventionellen Schreibweise ganz links. Bei einer 8-Bit-Zahl ist das Bit 7 das MSB mit dem Stellenwert $2^7 = 128$. Das MSB bestimmt damit maßgeblich die Größe einer vorzeichenlosen Binärzahl.